PungentFunk Utilities

Package Split, Utility Allocation, and Extension Map

Static source audit edition - generated from PungentFunkUtilities.zip

Date: 2026-05-10

Purpose. This document translates the updated package-modularity and Authoring Foundation doctrine into a practical package map. It also audits the uploaded source archive to identify current utilities, extension ownership, dependency boundaries, and extraction work required before the suite can ship as modular optional packages.

Audit basis. The uploaded archive contains 540 files, including 225 C# scripts. It currently uses two asmdefs only: PungentFunk.Utilities.Runtime and PungentFunk.Utilities.Editor. Therefore, the package split described here is a target architecture, not an already-enforced assembly/package layout.

Key conclusion. The source already contains many of the right logical systems: a central utility registry, categories, minimizer, theme, help browser, shared scan contracts/cache, Notes/Tokens, and an initial Authoring foundation with IDs, metadata, targets, references, validation models, provider contracts, provider registry, previews, actions, and missing-extension placeholders. The largest gap is not missing utility concepts; it is that package boundaries are still monolithic and Core currently hard-references concrete extension windows/actions.

1. Executive package map

Package	Tier	Should contain	Primary boundary note
Free / Core	Free/core	Registry, launcher, shared theme/prefs, minimizer, help/docs read/open contracts, scan contracts/cache shell, authoring contracts, provider registry, lightweight authoring browser shell.	Core must not compile against concrete lab windows. Current source still does, so extraction requires registrar split.
Documentation & Authoring	Paid extension with some free browser functionality in Core	Rich Document Editor, advanced Authoring Browser, Notes/Roadmap migration, help authoring, documentation-link editing, templates, text/token authoring helpers.	Current Notes, Tokens, and Authoring provider code should be split into contracts, concrete providers, and bridges.
Board / Graph / Visualization	Paid extension	Whiteboard/board documents, configurable node graph editor, graph widgets, relationship/dependency maps, minimap, blackboard data.	No concrete board editor scripts in this archive; Authoring foundation already reserves Board targets/actions.
Spreadsheet / Data Sheet	Paid extension	Data sheet model, table editor, CSV import/export, scanner-generated sheets, token/content sheets, Coverage Matrix successor.	Coverage Matrix is the current predecessor and should become a Data Sheet view/compatibility route.
Audit / Validation /	Paid extension;	Design Validation Audit	Core should keep scan

Scanning	developer/internal subfeatures allowed	advanced UI, project audit providers, reference scanner, terrain scanner, token validator advanced scan features, release readiness, findings actions.	contracts only. Current Project Audit window lives in Core folder and should move.
Scene Authoring / Placement	Paid extension	Asset Placement Lab, Surface Align, Modular Path Builder, Scene Navigation, Scene Gizmo Browser, Area Authoring Toolkit, scene gizmo source, placement repair/context tools.	Strong current script coverage; should get its own runtime/editor asmdefs.
Asset Production	Paid extension	Prefab Icon Generator, Prefab Asset Exporter, Font Preview, Texture Array Baker, preview/export queues.	Currently mostly editor-only; Texture Array Baker belongs here even if texture generator stays in Appearance.
Appearance / Colour / Texture	Paid extension with Core theme bridge	Palette Designer, colour analysis/accessibility, theme preview integrations, procedural texture generation where appearance-driven.	Theme customizer remains Core; palette/colour lab and procedural texture generator move here.
Audio Authoring	Paid extension	Audio Setup Coverage, Audio Catalog Coverage, audio SOs, contact audio routing/classification, terrain/surface audio profiles, audio validation providers.	Contains runtime audio models plus editor coverage tools; needs runtime/editor split.
Debug / Diagnostics	Paid or developer-focused extension	Debug Controller, DebugRouter, DebugSignalRelay, scheduled actions, signal/channel diagnostics, reflected component state panels.	Runtime debug router and editor controller should ship together with developer visibility controls.
UI / Input Feedback	Paid extension or small companion	Input Prompt Icon Library Populator, input prompt/token bridges, future UI feedback presenters.	Currently one concrete utility plus future bridge potential.
Content Generation	Paid extension	Name Generator, MadLib/name list assets, future text generation helpers, document/content export bridge.	Current name generation assets and editor are ready to become an independent package.
Full Bundle	Paid bundle	Everything above plus all official bridges, samples, dashboards, and coordinated workflows.	Must not be the only configuration that compiles or works.

2. Current source audit summary

- 225 C# scripts were found across runtime folders and Editor subfolders.
- Only two asmdefs currently enforce compilation boundaries: one Runtime asmdef and one Editor asmdef.
- The registry currently registers 42 utility/action descriptors.

- Existing product-facing categories already line up with the proposed packages: core-workflow, appearance-styling, scene-authoring, asset-production, generation-design, audio-authoring, audit-validation, batch-repair, debug-diagnostics, documentation-planning, ui-input-feedback, and scanning-coverage.
- Authoring foundation files already exist under Authoring/ and Editor/Authoring/. They should become the shared basis for notes, documents, boards, sheets, tasks, help topics, documentation links, tokens, audit issues, utilities, and external references.
- Shared scan files already exist under Editor/Utilities Core/Scanning/. They should remain contract/cache/state helpers in Core, while scan provider implementations and advanced scan UIs move to Audit / Validation or domain packages.

Finding	Source signal	Package implication
Only two asmdefs	The archive has one runtime asmdef and one editor asmdef.	Package boundaries are logical only. Create package-specific runtime/editor asmdefs before shipping modular packages.
Core registry hard-references lab windows	PungentUtilityRegistry imports editor namespaces for Audio, Colour, Content, Debugging, Generation, Placement, PreviewExport, ProjectAudit, SceneTools, and UI.	Move registration into per-package registrars. Core registry stores descriptors and exposes Register/RegisterRange only.
Core menu owner hard-references extensions	PungentUtilityToolMenus references PrefabAssetExporter and InputPromptIconLibraryAutoFill directly.	Move package-specific Tools menu entries into extension menu classes. Core keeps generic Utilities Browser/tray/theme/help entries.
Core access menu adds scene component	PungentUtilityAccessMenus adds PungentSceneGizmoSource directly.	Move Add Scene Gizmo Source action/menu to Scene Authoring package.
Descriptor metadata is incomplete for modular packaging	PungentUtilityDescriptor has PackageStatus but no packageId, dependency list, capabilities, extension points, bridge state, or fallback behavior.	Add package metadata fields and validation.
Authoring bootstrap is centralised	PungentAuthoringProviderBootstrap registers note, documentation, utility, token, help, and audit providers together.	Keep base registry in Core. Move concrete provider registration to owning package or bridge.
Coverage Matrix is not yet a Data Sheet	Current tool remains a standalone audit/planning window.	Deprecate as compatibility UI and build Data Sheet successor around shared sheet model.
Board/Data Sheet editors are placeholder-only	Authoring model reserves Board/DataSheet IDs and missing-extension actions, but no concrete editor packages exist.	Begin with contract-compliant MVPs after Authoring Browser shell is stable.

3. Detailed package definitions

Free / Core

Field	Recommendation
Package ID	com.pungentfunk.utilities.core
Tier	Free/Core
Required dependencies	Unity Editor baseline; no concrete extension dependencies.
Optional dependencies / bridges	All extension packages, but only through registry/service/provider discovery.
Boundary note	Do not keep concrete lab windows, package-specific menu items, or package-specific providers in Core.

	Current registry/menu files must be split into Core registry plus extension registrars.

Utilities and extensions to contain:

- Utility registry and descriptors
- Utilities Browser / launcher
- Utility Window Theme and preferences
- Minimized Utilities / tray
- Core Help Browser read/browse support
- Documentation link read/open contracts
- Developer Mode visibility contracts
- Shared scan contracts, cache shell, summaries, severities, GUI primitives
- Shared Authoring Data Foundation contracts and provider registry interfaces
- Lightweight Authoring Browser shell if kept free

Registered utility/action IDs:

- utilities-browser
- theme-customizer
- help-browser
- utility-tray
- minimize-focused-window
- open-minimized-utilities-tray
- utility-metadata-editor
- category-reference-membership

Documentation & Authoring

Field	Recommendation
Package ID	com.pungentfunk.utilities.authoring
Tier	Paid extension; reduced browser shell may remain free/core
Required dependencies	Core.
Optional dependencies / bridges	Audit / Validation, Data Sheet, Board / Graph, Content Generation, UI/Input Feedback, Token contracts.
Boundary note	Preserve legacy notes and integrations. Deprecate only the old body editing panel; route real editing to Rich Document Editor.

Utilities and extensions to contain:

- Rich Document Editor
- Advanced Authoring Browser
- Legacy Notes / Roadmap manager
- Help Browser authoring extensions
- Documentation Links editing tools
- Document templates and release/brief/debug-log templates
- Token insertion helpers
- Game-text authoring templates
- Legacy note migration tools

Registered utility/action IDs:

- tooltip-notes
- documentation-links (editing portion)

- future rich-document-editor
- future authoring-browser-advanced

Board / Graph / Visualization

Field	Recommendation
Package ID	com.pungentfunk.utilities.board
Tier	Paid extension
Required dependencies	Core Authoring contracts.
Optional dependencies / bridges	Documentation & Authoring, Audit / Validation, Data Sheet.
Boundary note	No concrete board editor exists in the archive. Authoring models already reserve Board item/target kinds and missing extension actions.

Utilities and extensions to contain:

- Whiteboard / board documents
- Configurable node graph editor
- Relationship/dependency map views
- Graph minimap and navigation widgets
- Blackboard node data
- Board card previews for authoring items

Registered utility/action IDs:

- future board-document-editor
- future whiteboard-editor
- future node-graph-editor
- future relationship-map

Spreadsheet / Data Sheet

Field	Recommendation
Package ID	com.pungentfunk.utilities.datasheet
Tier	Paid extension
Required dependencies	Core Authoring contracts.
Optional dependencies / bridges	Audit / Validation, Token contracts, Documentation & Authoring, Content Generation.
Boundary note	Coverage Matrix should be deprecated into this package instead of remaining an isolated audit/planning window.

Utilities and extensions to contain:

- Data sheet model
- Table editor
- CSV import/export
- Scanner-generated sheets
- SerializedProperty binding/adapters
- Token/content sheets
- Coverage Matrix successor

Registered utility/action IDs:

- coverage-matrix as compatibility route
- future data-sheet-editor
- future csv-import-export

- future scanner-generated-sheets

Audit / Validation / Scanning

Field	Recommendation
Package ID	com.pungentfunk.utilities.audit
Tier	Paid extension; developer/internal controls allowed
Required dependencies	Core scan contracts/cache and Core registry.
Optional dependencies / bridges	Documentation & Authoring, Data Sheet, Scene Authoring, Audio, Debug.
Boundary note	Core should own scan/result contracts only. Domain providers and advanced project audit UI belong here or in domain extensions with optional audit registration.

Utilities and extensions to contain:

- Design Validation Audit advanced UI
- Project audit providers
- Reference Assignment Scanner
- Terrain Usage Scanner
- Token Validator advanced scan UI
- Release readiness checks
- Findings actions
- Batch & Repair submodule for Component Matcher

Registered utility/action IDs:

- design-validation-audit
- reference-assignment-scanner
- terrain-usage-scanner
- component-tuning-copy
- token-validator advanced UI

Scene Authoring / Placement

Field	Recommendation
Package ID	com.pungentfunk.utilities.scene-authoring
Tier	Paid extension
Required dependencies	Core.
Optional dependencies / bridges	Audit / Validation, Board / Graph, Asset Production.
Boundary note	This is one of the strongest current packages because runtime and editor files are already grouped well by folder.

Utilities and extensions to contain:

- Asset Placement Lab
- Surface Align Tool
- Modular Path Builder
- Scene Navigation
- Scene Gizmo Browser
- Area Authoring Toolkit
- Placement sockets/groups/rules/asset sets
- Placement repair and context tools
- Scene gizmo source/editor

Registered utility/action IDs:

- asset-placement-lab
- create-placement-asset-set-from-selection
- add-placement-socket
- surface-align
- modular-path-builder
- scene-navigation
- scene-gizmo-browser
- fast-travel-sceneview-to-selection
- add-scene-gizmo-source
- path-authoring-toolkit
- bulk-rename

Asset Production

Field	Recommendation
Package ID	com.pungentfunk.utilities.asset-production
Tier	Paid extension
Required dependencies	Core.
Optional dependencies / bridges	Appearance / Colour / Texture, Audit / Validation.
Boundary note	Keep mostly editor-only. Use optional bridge to Appearance for palette/theme previews and texture generator outputs.

Utilities and extensions to contain:

- Prefab Icon Generator
- Prefab Asset Exporter
- Font Preview
- Texture Array Baker
- Preview/export queues
- Asset export safety helpers

Registered utility/action IDs:

- font-preview
- prefab-icon-generator
- prefab-asset-exporter
- save-selected-prefab-with-mesh-assets
- texture-array-baker

Appearance / Colour / Texture

Field	Recommendation
Package ID	com.pungentfunk.utilities.appearance
Tier	Paid extension
Required dependencies	Core theme contracts.
Optional dependencies / bridges	Asset Production, Documentation & Authoring, UI/Input Feedback.
Boundary note	Core keeps Utility Window Theme. Palette Designer and procedural texture workflows live here and bridge back into Core theme only through soft capability providers.

Utilities and extensions to contain:

- Palette Designer
- Colour analysis and contrast tools
- Colour deficiency previews

- Palette generation/storage
- Theme preview integrations
- Procedural Texture Generator when appearance-driven

Registered utility/action IDs:

- palette-designer
- procedural-texture-lab
- editor-style-explorer if treated as appearance developer tool

Audio Authoring

Field	Recommendation
Package ID	com.pungentfunk.utilities.audio
Tier	Paid extension
Required dependencies	Core.
Optional dependencies / bridges	Audit / Validation, Data Sheet.
Boundary note	This package has runtime-facing audio models and editor coverage windows. Split into audio runtime/editor asmdefs.

Utilities and extensions to contain:

- Audio Setup Coverage
- Audio Catalog Coverage
- Audio clip sets
- Audio coverage profiles
- Interaction matrices/profiles
- Material library and terrain material profiles
- Contact audio router/classifier/loop models
- Audio validation providers

Registered utility/action IDs:

- audio-setup-coverage
- audio-catalog-coverage

Debug / Diagnostics

Field	Recommendation
Package ID	com.pungentfunk.utilities.debug
Tier	Paid extension or developer-focused extension
Required dependencies	Core.
Optional dependencies / bridges	Audit / Validation, Documentation & Authoring.
Boundary note	Good standalone developer package. Keep project-specific debug integrations outside generic package or in project adapters.

Utilities and extensions to contain:

- Debug Controller
- DebugRouter runtime
- DebugSignalRelay
- Debug channels
- Scheduled actions
- Signal/channel diagnostics
- Reflected component state panels

Registered utility/action IDs:

- debug-control

UI / Input Feedback

Field	Recommendation
Package ID	com.pungentfunk.utilities.ui-input
Tier	Paid extension or small companion
Required dependencies	Core.
Optional dependencies / bridges	Token contracts, Documentation & Authoring, Audit / Validation.
Boundary note	Current source contains one concrete utility. Future value comes from token and UI feedback bridges.

Utilities and extensions to contain:

- Input Prompt Icon Library Populator
- Input prompt/token bridges
- Future UI feedback runtime/editor presenters
- Input prompt validation providers

Registered utility/action IDs:

- input-prompt-icon-library
- populate-selected-input-prompt-library
- populate-default-input-prompt-library

Content Generation

Field	Recommendation
Package ID	com.pungentfunk.utilities.content-generation
Tier	Paid extension
Required dependencies	Core.
Optional dependencies / bridges	Documentation & Authoring, Data Sheet, Board / Graph, UI/Input Feedback.
Boundary note	Good lightweight standalone extension. Use bridges to write generated output into documents/sheets/boards.

Utilities and extensions to contain:

- Name Generator
- NameList and MadLibNameList assets
- Future text generation helpers
- Document/content export bridge
- Generator presets

Registered utility/action IDs:

- name-generator

4. Registered utility allocation table

Target package	Utility ID	Display name	Kind	Module	Recommendation
Free / Core	utilities-browser	Utilities Browser	Window	Core shell	Keep in Core. It should discover

					extension registrars rather than reference windows directly.
Free / Core	theme-customizer	Utility Window Theme	Window	Theme / Appearance	Keep Core because all windows consume shared theme/prefs. Palette integration should be optional.
Free / Core	help-browser	Help Browser	Window	Documentation	Keep read/browse in Core. Authoring/editing extensions belong in Documentation & Authoring.
Free / Core	utility-tray	Minimized Utilities	Window	Workflow	Keep in Core with minimize actions.
Free / Core	minimize-focused-window	Minimize Focused Editor Window	Action	Workflow	Keep in Core.
Free / Core	open-minimized-utilities-tray	Open Minimized Utilities	Action	Workflow	Keep in Core.
Free / Core / Developer	utility-metadata-editor	Utility Metadata Editor	Action	Registry metadata	Keep as developer-only Core authoring tool.
Free / Core / Developer	category-reference-membership	Category Reference / Membership	Action	Registry metadata	Keep as developer-only Core authoring tool.
Documentation & Authoring	documentation-links	Documentation Links	Window	Documentation	Split safe read/open into Core; move editing popup to Documentation & Authoring developer tools.
Audit / Validation / Scanning	design-validation-audit	Project Audit	Window	Audit	Move advanced UI out of Core. Keep scan/result contracts in Core.
Appearance / Colour / Texture / Developer	editor-style-explorer	Editor Style Explorer	Window	Style diagnostics	Developer-only; can live in Core developer tools or Appearance extension depending on release model.
Debug / Diagnostics	debug-control	Debug Controller	Window	Debugging	Move to Debug package with runtime DebugRouter/SignalRelay.
Scene Authoring /	asset-placement-lab	Asset Placement Lab	Window	Asset Placement	Move to Scene Authoring /

Placement					Placement.
Scene Authoring / Placement	create-placement-asset-set-from-selection	Create Placement Asset Set From Selected Prefabs	Action	Asset Placement	Move with Asset Placement Lab.
Scene Authoring / Placement	add-placement-socket	Add Placement Socket	Action	Asset Placement	Move with Asset Placement Lab.
Scene Authoring / Placement	surface-align	Surface Align Tool	Window	Placement Assist	Move to Scene Authoring / Placement.
Scene Authoring / Placement	modular-path-builder	Modular Path Builder	Window	Modular Paths	Move to Scene Authoring / Placement.
Scene Authoring / Placement	scene-navigation	Scene Navigation	Window	Navigation	Move to Scene Authoring / Placement.
Scene Authoring / Placement	scene-gizmo-browser	Scene Gizmo Browser	Window	Gizmos	Move to Scene Authoring / Placement.
Scene Authoring / Placement	fast-travel-sceneview-to-selection	Fast Travel SceneView To Selection	Action	Navigation	Move with Scene Navigation.
Scene Authoring / Placement	add-scene-gizmo-source	Add Scene Gizmo Source	Action	Gizmos	Move with Scene Gizmo Browser; current Core access menu directly adds this component and should be extracted.
Scene Authoring / Placement	path-authoring-toolkit	Area Authoring Toolkit	Window	Path Authoring	Move to Scene Authoring / Placement.
Appearance / Colour / Texture	palette-designer	Palette Designer	Window	Colour	Move to Appearance / Colour / Texture.
Appearance / Colour / Texture	procedural-texture-lab	Texture Generator	Window	Procedural Textures	Move here if appearance-driven; bridge to Asset Production for export queues.
Asset Production	texture-array-baker	Texture Array Baker	Window	Texture Assets	Move to Asset Production.
Audio Authoring	audio-setup-coverage	Audio Setup Coverage	Window	Setup Coverage	Move to Audio Authoring; register audit provider when Audit package installed.
Audio Authoring	audio-catalog-coverage	Audio Catalog Coverage	Window	Catalog Coverage	Move to Audio Authoring; optional Audit bridge.
Asset Production	font-preview	Font Preview	Window	Font Assets	Move to Asset Production with optional Appearance bridge.
Asset Production	prefab-icon-generator	Prefab Icon Generator	Window	Prefab Assets	Move to Asset Production.

Asset Production	prefab-asset-exporter	Prefab Asset Exporter	Window	Prefab Assets	Move to Asset Production.
Asset Production	save-selected-prefab-with-mesh-assets	Save Selected as Prefab With Mesh Assets	Action	Prefab Assets	Move with Prefab Asset Exporter.
Audit / Validation / Scanning	component-tuning-copy	Component Matcher	Window	Component Tools	Place in Audit / Validation as a Batch & Repair submodule.
Audit / Validation / Scanning	reference-assignment-scanner	Reference Assignment Scanner	Window	Reference Repair	Move to Audit / Validation.
Audit / Validation / Scanning	terrain-usage-scanner	Terrain Usage Scanner	Window	Terrain Assets	Move to Audit / Validation with optional Scene Authoring bridge.
Scene Authoring / Placement	bulk-rename	Bulk Rename	Window	Batch Authoring	Move to Scene Authoring or Batch & Repair submodule. Preferred initial home: Scene Authoring.
Documentation & Authoring	tooltip-notes	Notepad / Notes & Roadmap	Window	Notes	Convert into Authoring Browser / legacy note manager.
Spreadsheet / Data Sheet	coverage-matrix	Coverage Matrix	Window	Coverage	Deprecate into Data Sheet package as coverage sheet/matrix view.
Audit / Validation / Scanning	token-validator	Token Validator	Window	Text Validation	Split parser/contracts into Core; advanced scan UI in Audit; authoring insertion bridge in Documentation & Authoring.
UI / Input Feedback	input-prompt-icon-library	Input Prompt Icon Library Populator	Window	Input Prompts	Move to UI / Input Feedback.
UI / Input Feedback	populate-selected-input-prompt-library	Populate Selected Input Prompt Library	Action	Input Prompts	Move with Input Prompt utility.
UI / Input Feedback	populate-default-input-prompt-library	Populate Default Input Prompt Library	Action	Input Prompts	Move with Input Prompt utility.
Content Generation	name-generator	Name Generator	Window	Names	Move to Content Generation.

5. Source folder allocation

Target package	Current source folders/files	Current responsibilities	Split note
Free / Core	Editor/Utilities Core/* plus	Registry, utility descriptors,	Currently also contains

	selected Authoring runtime contracts	categories, menus, control panel, Developer Mode contracts, Help Browser core, minimized utilities, theme/prefs, scanning contracts/cache, authoring ID/metadata/reference/target/validation contracts.	concrete lab references and Project Audit UI. Extract lab registrars and advanced audit/editor tools.
Documentation & Authoring	Authoring/*, Editor/Authoring/*, Editor/Notepad/*	Authoring ID/kind/metadata/reference/target/validation models; provider contracts/registry/bootstrap; built-in providers; Notes, Tokens, help/audit/doc bridges.	Move pure runtime contracts to Core. Move concrete providers into owning packages or optional bridges.
Scene Authoring / Placement	Asset Placement Lab/*, Editor/Asset Placement Lab/*, Modular Paths/*, Editor/Modular Paths/*, Editor/Scene Gizmos/*, PungentSceneGizmoSource.cs, scene/path/navigation windows	Placement data/runtime components, surface brush, asset-set builder, placement apply/repair, modular paths, scene navigation, gizmo browser/source editor, area/path toolkit.	Needs own runtime and editor asmdefs. Core access menus currently reference scene gizmo runtime component.
Appearance / Colour / Texture	Palette Designer/*, Editor/Palette Designer/*, Generation/*, Editor/Procedural Texture Lab/*	Colour conversion/contrast/deficiency/harmony, palette analysis/generation/storage/editor panels, procedural texture generator/settings/window.	Theme remains Core. Procedural texture export may bridge to Asset Production.
Audio Authoring	Audio Coverage/*, Editor/Audio Coverage/*	Audio clip sets, coverage profiles, interaction matrices/profiles, material libraries, contact audio routing/classification, catalog/setup coverage windows/scanners/editors.	Runtime contact audio models should be isolated from editor coverage tools.
Debug / Diagnostics	Debug/*, Editor/Debug Control Window/*	Debug channels/router/signal relay, debug discovery, scheduled actions, component panels, router panels, reflection utilities.	Good candidate for one extension with runtime/editor asmdefs and optional Audit bridge.
Asset Production	Editor/Prefab Exporter/*, Editor/PrefabIconGeneratorWindow.cs, Editor/FontPreviewWindow.cs, Editor/Texture Array Baker/*	Prefab export, prefab icon generation, font preview, texture array baking.	Mostly editor-only; keep AssetDatabase operations isolated and explicit.
Audit / Validation / Scanning	Editor/ReferenceAssignmentScannerWindow.cs, Editor/TerrainUsageScannerWindow.cs, Editor/PungentCoverageMatrixWindow.cs, Editor/ComponentTuningCopyWindow.cs plus advanced project audit UI	Reference assignment scanning, terrain usage scanning, component copy, coverage matrix, design validation audit UI and providers.	Core scan contracts should remain in Core; provider implementations and windows move here.
UI / Input Feedback	Editor/InputPromptIconLibraryAut	Input prompt icon library population from sprite	Future token bridge and runtime/editor UI feedback

	oFill.cs	filenames/serialized fields.	presenters can extend this package.
Content Generation	Name Generator/*, Editor/Name Generator/*	Name list assets, MadLib name lists, name generator window/editors.	Good standalone extension; optional Authoring bridge for writing generated text into documents.

6. Optional bridge architecture

Rule. A bridge should be a separate package or asmdef-scoped integration layer that can be removed without breaking either side. Bridges register providers, menu handoffs, exports, previews, and actions only when both dependencies are installed.

Bridge	Purpose	Dependencies	Implementation note
Authoring <-> Audit	Create notes/documents/tasks from findings; show authoring backlinks in findings; export audit result narratives.	Documentation & Authoring, Audit / Validation	Move PungentAuthoringAuditIssueProvider into a bridge or Audit-owned registrar.
Authoring <-> Token	Insert tokens into documents; validate token usage inside notes/rich docs; link token definitions to authoring targets.	Documentation & Authoring, Token core/Audit validator	Parser contracts may be Core; advanced validator remains Audit.
Audit <-> Data Sheet	Export findings, coverage gaps, token warnings, and scanner results into sheets.	Audit / Validation, Data Sheet	Coverage Matrix becomes a compatibility route.
Board <-> Authoring	Drop notes/documents/tasks as cards; open source item from node; preview backlinks.	Board / Graph, Documentation & Authoring/Core Authoring	Use shared PungentAuthoringReference/Target.
Board <-> Audit	Render findings/dependencies as graphs; create dependency maps from scan results.	Board / Graph, Audit / Validation	Optional graph bridge.
Data Sheet <-> Token	Token inventory sheets, content coverage sheets, validation dashboards.	Data Sheet, Token core/Audit validator	Optional provider; no hard editor dependency.
Colour <-> Theme	Use palettes to preview or generate UtilityWindowTheme presets.	Appearance / Colour, Core theme	Theme customizer should not hard-reference Palette Designer.
Scene Authoring <-> Audit	Placement/path/gizmo validation providers and repair actions.	Scene Authoring, Audit / Validation	Scene providers register only when Audit installed.
Audio <-> Audit	Audio coverage findings exposed in project scan UI and sheets.	Audio Authoring, Audit / Validation	Current audio coverage windows can become provider consumers.
Content Generation <-> Authoring	Generate names/text into documents, notes, sheets, or board cards.	Content Generation, Documentation & Authoring/Data Sheet/Board	Use action providers rather than direct window calls.
UI/Input <-> Token	Input prompt tokens and icon library validation.	UI / Input Feedback, Token core/Audit validator	Current Input Prompt utility can expose a provider.

7. Required package metadata additions

Current PungentUtilityDescriptor metadata is useful for browser display, but insufficient for package splitting. Add the following fields or an attached package metadata object:

- packageId and packageDisplayName
- packageTier: Core, Extension, Bridge, Bundle, ProjectAdapter, Internal
- packageOwner and moduleId
- requiredPackageIds and optionalPackageIds
- providedCapabilities and consumedCapabilities
- extensionPointsProvided and extensionPointsConsumed
- bridgeId and bridgeAvailabilityState
- missingDependencyMessage and fallbackBehavior
- minimumCompatibleVersion and documentationTopicId
- isDeveloperOnly, isExperimental, and isBundleOnly release flags

Capability family	Examples
Core workflow	registry, launcher, theme, prefs, minimizer, help browse, docs open, scan contracts, authoring contracts
Authoring	authoring item provider, preview provider, open/edit/copy actions, conversion actions, migration hooks
Scanning	scan provider, background-safe provider, cache provider, finding action provider, data-sheet export provider
Scene authoring	scene handles, scene overlay, path authoring, placement brush, gizmo source, surface alignment
Asset production	preview generation, export queue, prefab export, texture array bake, generated asset safety
Bridge state	installed, installed-but-disabled, missing-required-package, version-mismatch, developer-mode-only, unsupported-context

8. Extraction sequence

1. Freeze the current monolithic package as the Full Bundle baseline for regression testing.
2. Introduce package metadata fields on PungentUtilityDescriptor and add package split validation to the Project Audit tooling.
3. Move PungentUtilityRegistry registration data out of Core into per-package registrar classes. Core keeps only the registry service and Core descriptors.
4. Create asmdefs for Core.Runtime, Core.Editor, Authoring.Runtime, Authoring.Editor, SceneAuthoring.Runtime, SceneAuthoring.Editor, Audio.Runtime, Audio.Editor, Debug.Runtime, Debug.Editor, and editor-only extension packages.
5. Split Authoring contracts from concrete authoring providers. Core keeps ID/metadata/reference/target/validation contracts and registry interfaces; packages own providers.
6. Convert Notes & Roadmap into Authoring Browser behavior, preserving legacy notes and integrations while replacing body editing with preview/editor-launch actions.
7. Move Design Validation Audit advanced UI and scanner/provider implementations into Audit / Validation while keeping scan contracts/cache in Core.
8. Extract Scene Authoring / Placement first because it has strong runtime/editor grouping and clear current utility coverage.
9. Extract Asset Production, Appearance / Colour / Texture, Audio, Debug, UI/Input, and Content Generation packages in that order.
10. Add optional bridges only after standalone extension packages compile and validate without each other.

9. Package validation matrix

Configuration	Validation expectation
Core only	Compiles with no lab windows. Utilities Browser opens. Theme, tray, help browse, scan contracts, and authoring contracts are usable. Missing extension actions are disabled with clear messages.
Core + Documentation & Authoring	Authoring Browser opens, legacy notes browse correctly, rich document editor opens if installed, old note body editing is not the primary workflow.
Core + Scene Authoring	Placement/path/gizmo/navigation tools compile and register without Audit, Audio, Colour, or Asset Production installed.
Core + Audit	Advanced audit UI opens using Core scan contracts. Missing domain providers are shown as absent, not broken.
Core + Data Sheet	Sheet editor and CSV support compile without Audit; scanner-generated sheet actions are disabled until Audit bridge exists.
Core + Board	Board editor compiles and can preview generic authoring references without Documentation & Authoring installed.
Full Bundle	All utilities and bridges work, but tests must not rely only on this configuration.

10. Immediate recommended Codex briefs

Brief	Objective	Why it comes next
Package metadata descriptor pass	Add packageId, tier, dependency, capability, bridge, and fallback metadata to descriptors and overrides.	Everything else relies on package ownership being visible and auditable.
Registry registrar split pass	Move extension registrations out of PungentUtilityRegistry into per-package registrars.	This removes the most important Core-to-lab dependency leak.
Menu ownership split pass	Move package-specific Tools/Assets/GameObject menus out of Core menu classes.	Prevents Core from referencing extension implementation types.
Authoring provider ownership pass	Keep provider registry/contracts in Core; move concrete providers to package registrars or bridges.	Prepares Authoring Browser, Rich Documents, Boards, and Data Sheets for concurrent work.
Asmdef/package skeleton pass	Create package-specific runtime/editor asmdefs and migrate folders gradually.	Turns the package map into enforced compile boundaries.
Authoring Browser conversion pass	Refactor Notes & Roadmap into read/preview/manage/launch browser behavior.	Preserves existing value while ending investment in the old writing panel.

11. Final recommendation

Proceed with package splitting as a staged architecture hardening effort, not a rewrite. The current archive already supports the conceptual split: utilities are categorized, the registry is centralized, shared scans exist, and the Authoring foundation is partially implemented. The next work should focus on making those boundaries enforceable: per-package registrars, package metadata, bridge-safe provider registration, and asmdef/package skeletons. Only after this should Rich Documents, Boards, and Data Sheets proceed in parallel.